

Executable Archaeology: Reanimating the Logic Theorist from its IPL-I and IPL-V Sources

David Moews and Jeff Shrager

Abstract—The Logic Theorist (LT), created by Allen Newell, J. C. Shaw, and Herbert Simon in 1955–1956, is widely regarded as the first artificial intelligence program. Its history spans several distinct incarnations as the underlying Information Processing Language (IPL) evolved: the original 1956 IPL-I “pseudocode,” which was only ever executed by hand; the IPL-II version that ran on the JOHNNIAC; and Stefferud’s 1963 pedagogical recoding in IPL-V. Here we describe two independent reanimations that bracket this history. Moews implemented an interpreter for the IPL-I abstract machine and, after minor repairs, executed the original 1956 listing—to our knowledge the first time the IPL-I Logic Theorist has ever been run by a machine. Shrager implemented a complete IPL-V interpreter in Common Lisp and executed the code transcribed directly from Stefferud’s 1963 RAND report, which proved 16 of 23 attempted theorems from Chapter 2 of *Principia Mathematica*—results historically consistent with the original system’s behavior. Comparing the two reanimations makes empirically visible the gap between the 1956 cognitive specification, which depended on the intelligence of its human simulators, and the engineering required to make a real machine prove theorems end-to-end.

Index Terms—Logic Theorist, IPL-I, IPL-V, artificial intelligence history, software archaeology, A. Newell, H. A. Simon, J. C. Shaw, list processing, digital preservation

I. INTRODUCTION

In the summer of 1956, a small group of researchers gathered at Dartmouth College for a workshop on what John McCarthy had dubbed “artificial intelligence.” Among them were Allen Newell and Herbert Simon, who arrived with something none of the other attendees had: a program—albeit one not yet running on any machine—that could discover proofs of mathematical theorems.¹

The Logic Theorist (LT) [1], [2] would ultimately prove 38 of the first 52 theorems in Chapter 2 of Whitehead and Russell’s *Principia Mathematica* [5], in some cases finding proofs more elegant than those produced by the human authors. For Theorem 2.85, LT discovered a shorter, more direct proof than the one published in *Principia*. Simon showed the new proof to Bertrand Russell himself, who “responded with delight” [14, p. 167]. Edward Feigenbaum, then an undergraduate taking a graduate course from Simon, recalls the moment the project’s

significance was first announced. As he later told Pamela McCorduck: “It was just after Christmas vacation—January 1956—when Herb Simon came into the classroom and said, ‘Over Christmas Allen Newell and I invented a thinking machine.’ And we all looked blank” [14, p. 138].

Beyond reproducing *Principia* proofs, the Logic Theorist introduced heuristic search strategies—means–ends analysis and subgoal decomposition—derived from protocol analyses of human problem solving. These ideas later became foundational in both artificial intelligence and cognitive architectures such as the General Problem Solver (GPS) [18], Soar, ACT, and EPAM [19]. LT thus sits at the very beginning of the fusion of AI and cognitive psychology.

LT was implemented in the Information Processing Language (IPL), a language created by the same team—Newell, Shaw, and Simon—specifically for building cognitive models [6]. IPL introduced list manipulation, dynamic memory allocation, symbols as first-class objects, recursion, and higher-order functions, and was an explicit, acknowledged ancestor of Lisp [7] and of other early list-processing systems such as Weizenbaum’s SLIP. Yet while Lisp has thrived for nearly seven decades, IPL has been extinct since the mid-1960s. Although numerous IPL-V implementations existed in the 1960s on various machines, no maintained implementations appear to be available in modern environments. The only prior modern attempt known to the authors is Richman’s unpublished IPL-V interpreter, written in Microsoft BASIC, which he used to run EPAM IV in the early 1990s [20].²

This paper describes two reanimations of the Logic Theorist, undertaken independently and only later discovered to be complementary. Moews implemented an interpreter for IPL-I—the 1956 “pseudocode” in which LT was first published, and which was never implemented on any machine—and, after minor repairs to the printed listing, executed the original 1956 program. Shrager implemented a complete IPL-V interpreter in Common Lisp and executed the LT code transcribed directly from Stefferud’s 1963 RAND technical report [11], in which more than 99% of the running code is the original code from that report. Following the convention that the version number tracks the IPL generation, we will call the 1956 IPL-I program *LT1* and the 1963 IPL-V program *LT5*.

This work illustrates a methodological approach that might be called *executable archaeology*: the study of historically significant computational systems through faithful reconstruction and execution of their original code. Early AIs were not merely

Authorship is alphabetical; the authors contributed independently and equally, as described in the text.

D. Moews: dmoews@fastmail.fm.

J. Shrager is with Bennu Climate, Inc.: jshrager@gmail.com.

¹It is worth noting that Art Samuel’s checkers-playing program existed a couple of years before LT, and it too was heuristic. But Samuel’s goal was to build a checkers player, whereas Newell, Simon, and Shaw’s explicit goal was to model human cognition across many domains. Samuel’s effort was brilliant in its own right—he also built arguably the first machine learning program, and the first system to learn from self-play.

²Richman wrote to the second author (personal communication, Sep 26, 2025, quoted with permission): “I just interpreted IPL-V in Microsoft Basic using the IPL-V manual that I found at the CMU library.”

theoretical proposals but functioning programs (or, in the case of LT1, hand-executed procedures) whose detailed behavior cannot be fully understood from published descriptions alone. Reanimating such systems provides a new empirical window into the origins of artificial intelligence; we return to exactly what that window shows in Section VI.

In prior work, a team including the second author restored Weizenbaum’s original ELIZA to operation on a reconstructed CTSS running on an emulated IBM 7094 [12]. Although the Logic Theorist and ELIZA addressed very different problems—formal theorem proving versus natural-language conversation—both emerged from the same symbolic, list-processing programming lineage originating in IPL. However, whereas the ELIZA work rested on a stack that emulated vintage hardware and operating systems, the present reanimations emulate the abstract IPL-I and IPL-V machines.

II. HISTORICAL BACKGROUND

A. The Origins and Versions of the Logic Theorist

The story of the Logic Theorist begins in the early 1950s at the RAND Corporation. Herbert Simon and Allen Newell, collaborating with J. C. (Cliff) Shaw, sought to simulate complex thought by manipulating symbols. However, the history of the program is characterized by a series of shifting specifications across different versions of the Information Processing Language.

The system described in the famous 1956 RAND report (P-868) [1], and in nearly identical form in the IRE Transactions that September [2], was a “paper” version, written in what was called IPL-I or the “Logic Language” (LL). IPL-I was never implemented on any machine: it was a pseudocode for an imagined abstract machine, executed entirely by hand. At the time of the Dartmouth workshop in the summer of 1956, Newell and Simon noted that the program was “essentially specified” and that hand simulations suggested it would prove the 60-odd theorems of Chapter 2 of *Principia Mathematica*. These hand simulations famously employed “the human computer”: Simon’s wife, his children, and graduate students acting as program components [8, Ch. 13].

The first version to run on real hardware was written in IPL-II for the RAND JOHNNIAC [4], [13], producing its first machine proof in August 1956 [4], [8]. The widely cited result that LT proved 38 of the first 52 theorems in *Principia*—including the discovery of a more elegant proof for Theorem 2.85 than the one produced by Whitehead and Russell—stems from the empirical explorations reported in 1957 [3].

The IPL-V version used in Shrager’s reanimation, transcribed from Stefferud’s 1963 memorandum, represents a third distinct phase. The program was converted to IPL-V by Fred Tonge and further developed by Stefferud as a pedagogical recoding of the original heuristic logic into the first version of the language made widely available for public use [11]. Thus the two reanimations described here bracket the program’s history: LT1 is the 1956 conceptual model exactly as first published, while LT5 is the 1963 “modernized” production of that model.

TABLE I
EVOLUTION OF THE LOGIC THEORIST AND IPL

Year	Version	Platform	Historical Milestones
1956	IPL-I (LL)	Manual	Conceptual spec; hand-simulated only.
1956–57	IPL-II	JOHNNIAC	First machine proofs; 38 of 52 theorems.
1958	IPL-IV	IBM 704	Internal RAND use; transition to IPL-V.
1963	IPL-V	Various	Stefferud’s pedagogical LT implementation.
2026	IPL-V	Lisp	Reanimation of LT5 (Stefferud’s source).
2026	IPL-I	Python	First machine execution of LT1.

B. IPL: The Invention of Symbol- and List-Processing

To implement LT, Newell, Shaw, and Simon needed a programming language suited to symbolic reasoning. No such language existed, so they created the “Information Processing Language.” As noted, IPL went through several iterations. The earliest versions (IPL-I and IPL-II) powered the 1956 hand simulations and subsequent JOHNNIAC runs; IPL-V, released in the early 1960s [6], became the definitive public standard.

IPL-V runs on an abstract machine organized around *cells*. Each cell has an associated symbol. There is no single distinguished stack; every cell is potentially a stack of symbols that can be pushed and popped. Symbols are central to IPL-V: they are first-class objects, and every cell, data structure, routine, and control element is addressable by a symbol (its name). A set of control cells (H0–H5) governs execution: H0 serves as the parameter and result “communication” cell, H1 serves as both program counter and call stack, H2 is the free list for dynamic memory allocation, and H3 is the cycle counter. Ten working cells (W0–W9) provide scratch storage. Everything in IPL-V—data, routines, even the program itself—is a list of cells, and thus also a list of symbols. Routines are invoked by pushing their symbol (name) onto H1. The IPL machine interprets the operation indicated in that cell, and then continues to whatever is the indicated next cell. Elegantly, because H1 is both program counter and call stack, no special machinery is required to operate the machine aside from basic stack manipulation instructions.

The language’s built-in operations are called J-functions (because their names begin with “J”: J0, J1, J2, and so on). There are approximately 150 of these, implementing operations from basic list manipulation to “generator” (iterator) protocols. In the original IPL-V, many J-functions were themselves written in IPL, although some were implemented at the level of the underlying physical machine.

IPL-I, the language of the 1956 publication, is recognizably the same kind of machine seen through an earlier lens: an assembler-like notation for an abstract list-processing computer, with primitive operations playing the role later standardized as the J-functions. But because IPL-I was executed only by hand, its specification could afford to be looser, leaving to the human executor details that a real machine implementation would have to nail down. This difference turns out to be central to what the two reanimations jointly reveal (Section VI).

III. REANIMATING LT1: THE 1956 IPL-I PROGRAM

The source code for LT1 was published twice in 1956, in RAND paper P-868 [1] and in the IRE Transactions on Infor-

mation Theory [2]; the listings are almost identical. The code is written in IPL-I, an assembler-like language for an abstract machine that, as discussed above, was never implemented on non-human hardware. The listing comprises roughly 400 lines.

Moews transcribed the listing from the published report and implemented an interpreter for the IPL-I abstract machine in Python. The printed code contains typos and other problems and is not immediately runnable in its published form; it required minor patches and bug fixes, each documented in the repository’s source files [21]. After these repairs, the 1956 program runs. To our knowledge, this is the first time the IPL-I version of the Logic Theorist has ever been executed by a machine.

One omission in the published program is particularly telling. Although the IPL-I code is capable of finding individual proof steps, and of determining when enough steps have been found to constitute a proof, it contains no logic to print the steps or to assemble them into an actual proof. The human simulators, of course, needed no such logic—they could see the steps they had executed and write the proof down. To obtain proofs from the machine execution, the interpreter prints each potential proof step as it is found, and a separate program assembles the proof from the resulting set of steps and checks it [21].

A driver program imitates the 1957 “Empirical Explorations” [3] regime by running LT1 successively on each of the theorems of *Principia* Chapter 2, allowing it to use, in the proof of each theorem, all axioms and all previous Chapter 2 theorems, whether or not it managed to prove them. In addition to substitution and detachment, LT1 employs chaining, inferring $p \rightarrow r$ from $p \rightarrow q$ and $q \rightarrow r$.

IV. REANIMATING LT5: STEFFERUD’S 1963 IPL-V PROGRAM

A. Source Materials and Transcription

Unlike the ELIZA reanimation, which began with the dramatic discovery of original source code printouts in Weizenbaum’s archives at MIT [12], the source materials for the Logic Theorist were already publicly available. Two documents were indispensable for the LT5 work:

- 1) **Stefferdud (1963)**, *The Logic Theory Machine: A Model Heuristic Program*, RAND Corporation memorandum RM-3731 [11]. This report contains a complete listing of the LT program in IPL-V—nearly 3,000 lines—along with detailed descriptions of every aspect of LT’s workings, including numerous flow charts.
- 2) **Newell et al. (1964)**, *Information Processing Language-V Manual*, 2nd edition [6]. The definitive reference for IPL-V, including the semantics of all J-functions, cell structure conventions, and the generator protocol, as well as numerous student exercises and sample programs.

Both documents proved to be exceptionally well-written. There were, however, occasional ambiguities—places where the specification assumed familiarity with conventions of 1960s computing (such as BCD character encoding) that are no longer common knowledge.

The LT code was transcribed from Stefferud’s report into a spreadsheet by Shrager and Anthony Hay. Subsequently, Rupert Lane identified several transcription errors using his “gridlock” tool [16], software designed to accurately extract tabular code from PDF documents. Despite multiple people putting multiple sets of eyes on the code, a surprising number of small transcription errors persisted through numerous rounds of review.

The transcribed code was converted into an S-expression format (`.liplv` files) suitable for loading by the Lisp-based interpreter. Conveniently, Lisp comment characters work in these files, allowing corrections and annotations to be documented inline.

B. A New IPL-V Interpreter in Lisp

Shrager implemented a new IPL-V interpreter, in Common Lisp, to execute the Logic Theorist listing. The interpreter implements the symbolic data structures and execution model described in the IPL-V manual and supports the primitives required by the program. Although the earliest versions of the Logic Theorist ran on the RAND JOHNNIAC, by the time the Stefferud listing was published (1963) IPL-V had already been implemented on multiple machines. Accordingly, this reconstruction executes the program within a machine-independent IPL-V interpreter rather than attempting to reproduce a specific JOHNNIAC hardware environment.

There were several reasons for emulating the IPL-V abstract machine rather than the original hardware environment (as was done for ELIZA on the IBM 7094 and CTSS [12]). First, the code in question is IPL-V, and so is not JOHNNIAC-specific. Additionally, no thorough JOHNNIAC emulator exists, and in any case the JOHNNIAC IPL code is not known to survive. There does exist an IBM 7094 emulator that includes an IPL-V interpreter running under a variant of the IBSYS batch operating system [10], and this emulator includes demo scripts for IPL-V. However, that IPL-V interpreter is a binary with no source code and no documentation; an attempt to run LT on this platform failed early in execution with no useful information about the cause, and with no source code the failure was essentially undebuggable.

More fundamentally, a central goal of the project was not merely to get LT to run, but to *elucidate* IPL-V—to develop a deep understanding of the language, the machine model, and the infrastructure of the period. Writing a new interpreter was essential to that goal. IPL-V is defined as an abstract machine specification, not as a program for a specific computer; it was implemented on many different machines through the mid-1960s before being supplanted by Lisp.

The interpreter (`iplv.lisp`) implements the complete IPL-V abstract machine as described in the 1964 manual: the cell table, the control cells H0–H5, the working cells W0–W9, and the push-down mechanisms. Lists are stored as linked chains of cells, following the manual’s specification. It is worth noting that the push-down (stack) mechanism is itself implemented as linked lists within the IPL-V system. Push and pop operations, which appear to be hardware primitives of the abstract machine, are internally built on the same list

mechanism that underlies everything else in IPL-V, including data structures, routines, and IPL programs themselves; like Lisp, IPL-V is lists all the way down.

The core of the interpreter is a function, `ipl-eval`, that fetches and executes instructions by executing the list named in the program counter (H1). The evaluator is re-entrant—a capability trivially implementable in Lisp—supporting the recursive calls that IPL-V requires. The J-functions are implemented as Lisp functions. While the interpreter implements the full IPL-V machine, only the subset of J-functions required by LT (and a few test programs) is currently implemented, amounting to several dozen of the approximately 150 defined in the manual.

A complete run of LT5 attempting proofs of 23 theorems is available at [22]. This takes approximately 574,000 IPL machine cycles and creates over 576,000 symbols³. It finishes in a few seconds on a typical modern laptop. On the JOHNNIAC at RAND, in the mid 1950s, this likely would have required hours.

C. Debugging Against the Archival Record

The most difficult phase of debugging the LT5 reanimation began once the interpreter was stable enough that LT ran without crashing: the program would prove trivial theorems but fail on simple ones, fall into runaway recursions, or fail to recognize a completed proof—anomalies whose causes were far from their symptoms. This experience has an uncanny resonance with the original developers’ accounts. In an interview conducted by Pamela McCorduck for her pioneering history of AI, Cliff Shaw described the most daunting phase of troubleshooting as occurring late in development, when the system was beyond crashes and produced behavior that appeared superficially reasonable, leaving the developers in a quandary as to whether the results were valid consequences of the heuristic logic or actual bugs, in a memory that had become a churning and impenetrable web of interconnected lists [9]. Sixty years later, with the same program, Shrager faced the identical challenge. In this latter phase, large language models (Google’s Gemini and Anthropic’s Claude) were used as debugging assistants, performing the grinding work of reading thousands of lines of execution traces and tracking machine state across tens of thousands of cycles; a fuller account of that collaboration is beyond the scope of this paper.

One episode, however, bears directly on the methodology at issue here. The final substantive bug was localized to J74, the list-copying function, yet the Lisp implementation appeared to match the manual’s prose description; the specification itself seemed to support the buggy behavior. Resolution came not from documentation but from an artifact: “Simon’s J’s,” a set of original IPL-V punched cards from 1962 preserved by the Computer History Museum [15], representing the production implementation of the J-functions at the moment IPL-V was being standardized. Comparing the 1962 card-deck implementation of J74 with the modern one confirmed a hypothesized discrepancy, and a one-line patch made LT run completely correctly. The manual’s prose, a careful modern reading of it,

and inference from the modern code had all been insufficient; the original 1962 source was the ground truth.

V. WHAT THE LOGIC THEORIST PROVES

The Logic Theorist attempts to prove theorems in propositional logic using the five axioms of *Principia Mathematica*, Chapter 2. The notation used by the original authors is given in Table II.

TABLE II
LOGICAL NOTATION USED BY THE LOGIC THEORIST

Symbol	Meaning
$\supset$	Implication ($\rightarrow$)
$\vee$	Disjunction ($\vee$)
$\wedge$	Conjunction ($\wedge$)
$\neg$	Negation ($\neg$)
$\equiv$	Biconditional ($\leftrightarrow$)
$\doteq$	Definitional equality

LT employs several proof methods:

- **Substitution**—substitute variables in an axiom or previously proved theorem to match the goal.
- **Detachment** (modus ponens)—from $A \rightarrow B$ and A , conclude B .
- **Forward chaining**—apply substitution and detachment working forward from known theorems.
- **Backward chaining**—work backward from the goal, seeking premises that would yield it.
- **Sublevel replacement**—replace a subexpression using a known equivalence.

In LT5, each proof attempt is bounded by a settable effort limit in terms of IPL machine cycles. (In all examples below this limit is set to 20,000 cycles.) This is a soft cap: it prevents the system from *beginning* a new subproblem exploration once exceeded, but does not interrupt a search in progress.

Table III shows the axioms and definitions that are initially provided to LT5 for the runs described in the present paper. Importantly, as LT proves a given theorem, that theorem is treated as an axiom and can be used to support subsequent proofs. Thus, LT can be said, in a sense, to learn!

TABLE III
AXIOMS AND DEFINITIONS

Name	Axiom/Def
1.01	$((PIQ) \doteq (\neg PVQ))$
2.33	$((PVQVR) \doteq ((PVQ) VR))$
3.01	$((P * Q) \doteq (\neg (\neg (PV \neg Q)))$
4.01	$((P = Q) \doteq ((PIQ) * (QIP)))$
1.2	$((AVA) IA)$
1.3	$((BI (AVB))$
1.4	$((AVB) I (BVA))$
1.5	$((AV (BVC)) I (BV (AVC)))$
1.6	$((BIC) I ((AVB) I (AVC)))$

Table IV shows the results of the reanimated LT5 on the 23 theorems attempted: 16 are proved, and the remaining 7 are not—results that are historically consistent with the original LT’s known behavior, given an effort limit of 20,000 IPL machine cycles.

³Yeah, we were surprised too. It’s really just a coincidence!

TABLE IV
THEOREMS ATTEMPTED (LT5)

Name	Formula	Result
2.01	$(PI-P) I-P$	Proved
2.02	$QI (PIQ)$	Proved
2.04	$(PI (QIR)) I (QI (PIR))$	Proved
2.05	$(QIR) I ((PIQ) I (PIR))$	Proved
2.06	$(PIQ) I ((QIR) I (PIR))$	Proved
2.07	$PI (PVP)$	Proved
2.08	PIP	Proved
2.10	$-PVP$	Proved
2.11	$PV-P$	Proved
2.12	$PI--P$	Proved
2.13	$PV---P$	Proved
2.14	$--PIP$	Not proved
2.15	$(-PIQ) I (-QIP)$	Not proved
2.20	$PI (PVQ)$	Proved
2.21	$-PI (PIQ)$	Not proved
2.24	$PI (-PVQ)$	Proved
3.13	$(- (P*Q)) I (-PV-Q)$	Not proved
3.14	$(-PV-Q) I (- (P*Q))$	Not proved
3.24	$- (P*-P)$	Not proved
4.13	$P=--P$	Not proved
4.20	$P=P$	Proved
4.24	$P= (P*P)$	Proved
4.25	$P= (PVP)$	Proved

A representative proof trace is shown below. Here LT5 proves Theorem 2.01, $(PI-P) I-P$:

2.01	$(PI-P) I-P$
PROOF FOUND.	
GIVEN	*1.2 (AVA) IA
SUBSTITUTION	.0 $(PI-P) IP$
SUBLEVEL REPL	2.01 $(PI-P) I-P$
Q.E.D.	
EFFORT	LIMIT 20000 ACTUAL 5579
SUBPROBLEMS	LIMIT 50 ACTUAL 1
SUBSTITUTIONS	LIMIT 50 ACTUAL 2

The proof begins from Axiom *1.2, (AVA) IA, substitutes $-P$ for A to obtain $(-PV-P) I-P$, then applies sublevel replacement (using the fact that P implies PVP) to transform the antecedent, yielding the desired theorem. The Appendix depicts a more complex proof (4.25), and the original output, including both of these proofs, is at [22].

VI. WHAT EXECUTABLE ARCHAEOLOGY CONTRIBUTES

It is reasonable to ask what is learned by running this old code that could not be learned by reading it, or by reading the abundant secondary literature on the Logic Theorist. The present case suggests several answers.

It makes the gap between theory and implementation measurable. LT1 and LT5 are, in intent, the same program; in fact they differ by nearly an order of magnitude in size—roughly 400 lines of IPL-I against nearly 3,000 lines of IPL-V.⁴ The 1956 program is something close to a direct expression of Newell and Simon’s hypothesized cognitive operators for human theorem proving: a high-level specification of a theory

of cognition. It could afford to be that, because its “hardware” was human—intelligent, flexible, and able to silently fill in whatever the specification left unsaid. LT5 is what the same design became when Shaw and his successors had to make a real, literal-minded machine do the whole thing end-to-end. Reanimating both versions converts a familiar qualitative observation about early AI—that getting from idea to running program was most of the work—into something that can be measured, localized, and studied line by line.

It reveals exactly what the human computer was silently doing. The most striking single finding of the LT1 reanimation is an absence: the published 1956 program can find proof steps and can recognize that a proof exists, but contains no machinery to assemble or output the proof itself. No reader of the report appears to have remarked on this, and no reading of the secondary literature would surface it; it only becomes visible when a machine, rather than one of Simon’s students, executes the code and the proof fails to appear. The omission is itself historical evidence: it marks precisely a boundary of what Newell and Simon considered the *theory* (finding the proof) versus mere clerical work (writing it down)—work the human simulators performed without anyone needing to specify it.

It converts historical claims into checkable ones. The 1956 report claimed, on the basis of hand simulation, that the program would prove the theorems of *Principia* Chapter 2; the 1957 paper reported the famous 38-of-52 result for the JOHN-NIAC version. Until now, both claims rested on testimony and on archival output listings. The reanimations allow such claims to be tested directly: LT5 proves 16 of 23 attempted theorems under a fixed effort limit, a result consistent with the original reports, and LT1’s behavior on the same theorem set can now be examined for the first time. Moreover, both systems are now experimental instruments: effort limits, axiom orderings, and theorem sequences can be varied, and the consequences observed on the genuine historical artifact rather than on a modern paraphrase of it.

It shows that some semantics survive only in artifacts, not in documentation. The J74 episode (Section IV-C) is a small but pointed demonstration: the IPL-V manual’s prose description of a core primitive was consistent with an incorrect implementation, and the discrepancy could be resolved only by consulting the original 1962 punched-card source preserved by the Computer History Museum. For the history of computing this is a methodological moral: specifications and manuals, however well written, systematically under-determine the systems they describe, and executable reconstruction is one of the few ways to discover where.

It corrects the published record. Both source listings turn out to contain errors—typos and inconsistencies in the 1956 IPL-I listing that prevent it from running as printed, and subtle transcription hazards in working from the 1963 report—now documented in the respective repositories [21], [22]. These repairs, and the requirement that every repair be justified and recorded, are the “archaeology” in executable archaeology.

Finally, the two reanimations were undertaken independently, by authors unknown to each other, and converged on complementary reconstructions of adjacent stages of the same

⁴the counting convention (instructions? card images?).

program. This independence functions as a kind of replication: agreement between the two efforts on the program’s structure and behavior is evidence that the reconstructions reflect the historical system rather than either reconstructor’s assumptions.

VII. RELATED WORK

The reanimation of historical software systems has gained momentum in recent years. Most directly related is the ELIZA reanimation by Lane et al. [12], in which a team including the second author restored Weizenbaum’s original ELIZA to operation on a reconstructed CTSS running on an emulated IBM 7094. That project reconstructed the complete original hardware and operating system stack; the present work instead emulates abstract machine specifications in modern languages.

Reimplementing the Logic Theorist’s *algorithms* in a modern language is a relatively straightforward exercise—essentially an introductory AI course project, and one that has surely been done many times. A 1968 paper, for example, describes an implementation of LT in LISP 1.5 [17]. The present work is a fundamentally different undertaking: rather than rewriting LT’s logic in a modern language, it constructs faithful emulators of the IPL-I and IPL-V abstract machines and runs the original code, transcribed directly from the 1956 and 1963 reports. The distinction is significant: a Lisp reimplementation demonstrates that LT’s algorithms can be expressed in Lisp (which is unsurprising), while the present work demonstrates that the original IPL code, as written and published by Newell, Simon, and Shaw’s team, functions as described—and, moreover, enables experiments on the real, working systems of machine and program.

VIII. NEXT STEPS

The IPL-V interpreter developed for this project is not specific to the Logic Theorist. It implements the general IPL-V abstract machine and can, in principle, run any IPL-V program. Work is in progress on a corpus of early IPL-V AI programs. The most significant targets are the General Problem Solver (GPS) [18] and EPAM (Elementary Perceiver and Memorizer) [19]. EPAM source code is available in the Feigenbaum archives at Stanford, and archival work to locate complete source code for GPS is in progress. Now that proven IPL interpreters exist, running these programs should require substantially less effort than the initial construction of the interpreters themselves.

The IPL-I interpreter, repaired LT1 source, and proof-assembly tools are open sourced at <https://github.com/dmoews/logic-theorist>. The IPL-V interpreter, the transcribed LT5 code, and complete outputs as described in this paper are open sourced at <https://github.com/jeffshrager/IPL-V>.

A PERSONAL CODA

One of us (Shrager) studied under both Newell and Simon at Carnegie Mellon University in the 1980s, where his work focused on modeling the cognition of scientists and engineers. IPL and LT were long since history by that time, and neither came up in classes or conversation. But the intellectual

values Newell and Simon instilled—rigor about mechanism, respect for the complexity of cognition, and the conviction that thinking could be understood computationally—shaped the careers of their students. There is a certain closure in now studying Newell, Simon, and Shaw *as* scientist-engineers: reverse-engineering their design decisions, reconstructing their reasoning, and trying to understand how they invented AI and cognitive science. The subject of that training and its object have folded into one another.

ACKNOWLEDGEMENTS

The following people (listed in no particular order) contributed to the LT5 effort in ways ranging from code transcription to weekly encouragement: Anthony Hay, Arthur Schwarz, Leigh Klotz, David M. Berry, Rupert Lane, Amy Majczyk, Emily Davis, Ethan Ableman, and Paul McJones. Shrager is particularly grateful to Hay, Schwarz, Klotz, and Berry, who put up with weekly progress updates throughout the project and responded with encouragement that was always helpful and sometimes technically substantive. Rupert Lane’s gridlock software caught several transcription errors that had survived multiple rounds of human review.

We thank the Computer History Museum for preserving and making available “Simon’s J’s,” the original 1962 IPL-V J-function card deck, which proved essential during debugging. We also thank Alexander Magoun and David Walden for editorial guidance, including the suggestion to combine our independent efforts into this single account.

Finally, we acknowledge Allen Newell, Herbert Simon, and Cliff Shaw, whose work this paper merely re-runs.

Portions of the LT5 debugging effort, and portions of the drafting of this paper, were carried out with the aid of various LLMs, but all concepts and final editorial decisions were the authors’ alone.

REFERENCES

- [1] A. Newell and H. A. Simon, *The Logic Theory Machine – A Complex Information Processing System*, RAND Corp., Santa Monica, CA, USA, Rep. P-868, 1956.
- [2] A. Newell and H. A. Simon, “The logic theory machine—A complex information processing system,” *IRE Trans. Inf. Theory*, vol. 2, no. 3, pp. 61–79, Sep. 1956.
- [3] A. Newell, J. C. Shaw, and H. A. Simon, “Empirical explorations with the logic theory machine: A case study in heuristics,” in *Proc. Western Joint Comput. Conf.*, 1957, pp. 218–230.
- [4] A. Newell and J. C. Shaw, “Programming the logic theory machine,” in *Proc. Western Joint Comput. Conf.*, 1957, pp. 230–240.
- [5] A. N. Whitehead and B. Russell, *Principia Mathematica*. Cambridge, UK: Cambridge Univ. Press, 1910–1913.
- [6] A. Newell, et al. *Information Processing Language-V Manual*, 2nd ed. Englewood Cliffs, NJ, USA: Prentice-Hall, 1964.
- [7] J. McCarthy, “Recursive functions of symbolic expressions and their computation by machine, Part I,” *Commun. ACM*, vol. 3, no. 4, pp. 184–195, Apr. 1960.
- [8] H. A. Simon, *Models of My Life*. Cambridge, MA, USA: MIT Press, 1996.
- [9] Pamela McCorduck Collection, Carnegie Mellon University Archives.
- [10] R. Storey, “B7094” [Online]. Available: <https://github.com/Bertoid1311/B7094> (Accessed: Mar. 11, 2026).
- [11] E. Stefferud, “The Logic Theory Machine: A Model Heuristic Program,” RAND Corp., Santa Monica, CA, USA, Rep. RM-3731-CC, 1963.
- [12] R. Lane, A. Hay, A. Schwarz, D. M. Berry, and J. Shrager, “ELIZA Reanimated: Restoring the mother of all chatbots to one of the world’s first time-sharing systems,” *IEEE Ann. Hist. Comput.*, vol. 47, no. 1, pp. 8–23, 2025.

- [13] F. Gruenberger, “The History of the JOHNNIAC,” RAND Corp., Santa Monica, CA, USA, Rep. RM-5654-PR, 1968.
- [14] P. McCorduck, *Machines Who Think*, 2nd ed. Natick, MA, USA: A. K. Peters, 2004.
- [15] A. Bochanek, “Simon’s J’s,” Computer History Museum Blog, Sep. 26, 2012. [Online]. Available: <https://computerhistory.org/blog/simons-js/> (Accessed: Mar. 11, 2026).
- [16] R. Lane, “Gridlock: Accurate code extraction from PDFs,” 2024. [Online]. Available: <https://github.com/rupertl/gridlock>
- [17] J. S. Millstein, “The Logic Theorist in LISP,” *Comput. J.*, vol. 11, no. 1, pp. 39–44, Feb. 1968.
- [18] A. Newell, J. C. Shaw, and H. A. Simon, “Report on a general problem-solving program,” in *Proc. Int. Conf. Inform. Process.*, 1959, pp. 256–264.
- [19] E. A. Feigenbaum, “The simulation of verbal learning behavior,” in *Proc. Western Joint Comput. Conf.*, 1961, pp. 121–132.
- [20] H. B. Richman, J. J. Staszewski, and H. A. Simon, “Simulation of expert memory using EPAM IV,” *Psychol. Rev.*, vol. 102, no. 2, pp. 305–330, 1995.
- [21] D. Moews, “Recreation of the 1956 IPL-I version of the Logic Theorist theorem prover,” 2026. [Online]. Available: <https://github.com/dmoews/logic-theorist> (Accessed: Jun. 9, 2026).
- [22] J. Shrager, Output from the reanimated logic theorist, [Online]. Available: https://github.com/jeffshrager/IPL-V/blob/master/major_results/20260309_CompleteAndCorrect.drb (Accessed: Mar. 11, 2026).

APPENDIX

To illustrate a complex proof generated by the reconstructed Logic Theorist (LT5), the following derivation corresponds to theorem *4.25 of *Principia Mathematica*:

$$P = (P \vee P)$$

This states the idempotence of disjunction. In the LT representation, the equivalence operator is defined using implication and conjunction:

$$(P = Q) \equiv ((P \rightarrow Q) \wedge (Q \rightarrow P))$$

The Logic Theorist proof trace proceeds as follows:

- 1) **Expand the definition of equivalence:** LT begins by reducing the goal to a conjunction of two implications:

4.25 P=(PVP)
 .0 (PI(PVP)) * ((PVP)IP) 4.01 ,0 REPLACEMENT

Logic form:

$$P = (P \vee P) \Rightarrow (P \rightarrow (P \vee P)) \wedge ((P \vee P) \rightarrow P)$$

- 2) **Identify the first implication:** The system identifies a forward-chaining path to establish the first half of the conjunction:

PI(PVP) 4.20 ,0 FORWARD CHAINING

Logic form:

$$P \rightarrow (P \vee P)$$

- 3) **Find the proof via substitution:**

LT utilizes Theorem *2.20 from its memory as the basis for substitution: *Theorem 2.20 is not given as an axiom, but is a previously proved theorem that it remembered and applied here; See comment below.*

GIVEN 2.20 AI(AVB)
 SUBSTITUTION .0 PI(PVP)

Logic form:

The general theorem $A \rightarrow (A \vee B)$ is retrieved

Substituting $A := P$ and $B := P$ yields $P \rightarrow (P \vee P)$

- 4) **Complete the equivalence:** LT completes the chain by linking the proved implications back to the original goal:

GIVEN 4.20 A=A
 FORWARD CHAINING 4.25 P=(PVP)
 Q.E.D.0

Logic form:

Because both $P \rightarrow (P \vee P)$ and $(P \vee P) \rightarrow P$ are established, the equivalence $P = (P \vee P)$ holds.

Performance Metrics:

EFFORT	LIMIT 20000	ACTUAL 8727
SUBPROBLEMS	LIMIT 50	ACTUAL 2
SUBSTITUTIONS	LIMIT 50	ACTUAL 3

The successful execution of complex derivations such as this one, which use previously proved theorems, demonstrates the Logic Theorist's capacity for a primitive form of machine learning: by treating previously proved theorems, such as 2.20, as established "givens" for subsequent search paths, the system bootstraps its own knowledge base, mirroring the cumulative nature of algebraic reasoning.